TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

BE Programme | Year III / Part II

EMBEDDED SYSTEM (CT 655)

QUESTION BANK WITH ANSWERS

Chapter 2: Hardware Design Issues

COURSE SYLLABUS AT A GLANCE

Unit	Topic	Hours	Marks
1	Introduction to Embedded System	3	4
2	Hardware Design Issues	4	8
3	Software Design Issues	6	8
4	Memory	5	8
5	Interfacing	6	8
6	Real-Time Operating System (RTOS)	8	12
7	Control System	3	8
8	IC Technology	3	8
9	Microcontrollers in Embedded Systems	3	8
10	VHDL	4	8
	Total	**45**	**80**

Chapter 2 Topics (4 Hours | 8 Marks):

- 2.1 Custom Single-Purpose Processors — Design Process
- 2.2 Algorithm → FSMD → Datapath → Controller
- 2.3 RT-Level Custom Processors: State Machines with Datapath
- 2.4 Optimizing Custom Single-Purpose Processors
- 2.5 Optimization Techniques: State Merging, Operator Sharing, Concurrency, Pipelining
- 2.6 Introduction to General-Purpose Processors

Chapter 2: Hardware Design Issues

Q1. Design a custom single-purpose processor for computing LCM of two numbers a and b.	2082 Kartik (Back) [8]

ANSWER

Algorithm:

LCM(a,b) = (a × b) / GCD(a,b). First compute GCD using repeated subtraction, then compute LCM.

1. Load a → reg_a, b → reg_b
2. Save original: orig_a ← a, orig_b ← b
3. While a ≠ b: if a > b then a ← a − b, else b ← b − a
4. GCD ← a (when a = b)
5. LCM ← (orig_a × orig_b) / GCD
6. Output LCM, assert done

LCM Processor — FSMD State Table		
State	**Condition**	**Operation**
S0	start=1	reg_a←a, reg_b←b, orig_a←a, orig_b←b → go to S1
S1	reg_a = reg_b	GCD←reg_a, product←orig_a×orig_b → go to S4
S1	reg_a > reg_b	reg_a←reg_a−reg_b → stay S1
S1	reg_a < reg_b	reg_b←reg_b−reg_a → stay S1
S4	—	LCM←product/GCD → go to S5
S5	—	output←LCM, done←1 → go to S0

Datapath Components:

- Registers: reg_a, reg_b, orig_a, orig_b, product, GCD, LCM
- ALU operations: subtractor (a−b), comparator (a>b, a=b), multiplier (orig_a×orig_b), divider (product/GCD)
- MUX: selects feedback paths for reg_a and reg_b

Controller: FSM generating load enables and ALU select signals per state table above.

Q2. Develop algorithm; draw the state diagram and design FSMD for GCD of two numbers.	2081 Chaitra (Regular) | 2079 Chaitra (Back) | 2078 Chaitra (Regular) [8]

ANSWER

Algorithm (Euclidean Subtraction):

1. Load a → reg_a, b → reg_b
2. While reg_a ≠ reg_b: if reg_a > reg_b then reg_a ← reg_a − reg_b, else reg_b ← reg_b − reg_a
3. Output GCD = reg_a, assert done

GCD Processor — FSMD State Table		
State	**Condition**	**Operation**
S0	start=1	reg_a←a, reg_b←b → go to S1
S1	reg_a = reg_b	output←reg_a, done←1 → go to S0
S1	reg_a > reg_b	reg_a←reg_a−reg_b → stay S1
S1	reg_a < reg_b	reg_b←reg_b−reg_a → stay S1

State Diagram:

```
          start=1 / load a,b
   ┌──────────────────────────────┐
   ▼                              │
  [S0] ────start=1──────▶ [S1] ──┘
                                 │   ▲
     a=b / out,done=1            │   │ a≠b / subtract
           ▼                     │   │
          [Done]                 └───┘
```

Datapath: Two registers (reg_a, reg_b), one subtractor, one comparator (>, <, =), output register.

Controller: 2-state FSM (S0=idle, S1=compute). Outputs: ld_a, ld_b, sub_sel (which to subtract), done.

Q3. What is optimization? Explain optimization techniques used in custom single-purpose processor design.	**2070 Bhadra (Regular) | 2077 Ashwin (Back) [8]**

ANSWER

Optimization in Custom Processor Design:

Optimization is the process of improving a custom single-purpose processor's implementation to better meet design metrics — reducing area (cost), power, or critical path delay (performance) — without changing its function.

Optimization Techniques:

1. State Merging: Combine multiple states in the FSMD into fewer states when their operations can be performed in the same clock cycle. Reduces number of states → smaller controller FSM → faster operation.

Example: If S1 (check condition) and S2 (compute result for one branch) have no data dependency, merge them into one state.

2. Operation Chaining: Perform multiple operations in sequence within a single clock cycle (combinational chain). Eliminates intermediate registers. Reduces clock cycles but may increase critical path.

```
Without chaining: S1: A←B+C  S2: D←A*E   (2 clock cycles)
With chaining:    S1: A←B+C, D←A*E       (1 clock cycle, longer path)
```

3. Operator Sharing (Resource Sharing): If the same operation type (e.g., addition) is used in different states, use a single shared adder with MUX inputs rather than separate adders. Reduces hardware area and cost.

```
Without sharing: Adder1 (state 1), Adder2 (state 3)  → 2 adders
With sharing:    1 Adder + 2-to-1 MUX on inputs        → 1 adder
```

4. Concurrency (Parallelism): Identify independent operations and execute them simultaneously (in parallel) in the same state. Reduces total clock cycles needed.

```
Sequential:  S1: A←B+C  S2: E←F*G    (2 cycles, independent ops)
Concurrent:  S1: A←B+C AND E←F*G     (1 cycle, parallel execution)
```

5. Pipelining the Datapath: Insert pipeline registers within the datapath to allow overlapping execution of consecutive processor invocations. Improves throughput for repeated computations but does not reduce latency of a single computation.

6. Algorithm Optimization: Choose a better algorithm before hardware implementation. Example: Use Euclidean division GCD (a mod b) instead of subtraction-based GCD — far fewer iterations for large numbers.

Q4. Design a custom single purpose processor to compute x^n (x to the power n).	**2080 Chaitra (Regular) | 2075 Baisakh (Back) [8]**

ANSWER

Algorithm:

1. Load x → reg_x, n → reg_n
2. result ← 1

3. While reg_n > 0: result ← result × reg_x; reg_n ← reg_n − 1
4. Output result, assert done

x^n Processor — FSMD State Table		
State	**Condition**	**Operation**
S0	start=1	reg_x←x, reg_n←n, result←1 → go to S1
S1	reg_n = 0	output←result, done←1 → go to S0
S1	reg_n > 0	result←result×reg_x, reg_n←reg_n−1 → stay S1

Datapath:

- Registers: reg_x (base), reg_n (counter), result (accumulator)
- ALU: multiplier (result × reg_x), decrementer (reg_n − 1), comparator (reg_n = 0)
- Controller signals: ld_x, ld_n, ld_result, dec_n, done

Q5. Design a single-purpose processor that computes the Fibonacci number F(n). Optimize it.	**2080 Ashwin (Back) | 2074 Bhadra (Regular) [5+3]**

ANSWER

Algorithm (Iterative):

1. Load n → reg_n
2. If n = 0: output 0, done
3. If n = 1: output 1, done
4. a ← 0, b ← 1, i ← 2
5. While i ≤ n: c ← a + b; a ← b; b ← c; i ← i + 1
6. Output b, done

Fibonacci Processor — FSMD State Table		
State	**Condition**	**Operation**
S0	start=1	reg_n←n → go to S1
S1	reg_n = 0	output←0, done←1 → S0
S1	reg_n = 1	output←1, done←1 → S0
S1	reg_n ≥ 2	a←0, b←1, i←2 → go to S2
S2	i > reg_n	output←b, done←1 → S0
S2	i ≤ reg_n	c←a+b, a←b, b←c, i←i+1 → stay S2

Optimization: State Merging + Operator Sharing

- Share the adder between c←a+b and i←i+1 using MUX inputs — saves one adder.
- Merge check state with compute state (S2) — reduces state count from 3 to 2 (S0, S2).

- The assignment c←a+b, a←b, b←c can be pipelined if multiple Fibonacci values are needed in sequence.

Q6. Develop algorithm; draw the state diagram and design FSMD for computing the mean of n numbers.	**2081 Ashwin (Back) | 2076 Jestha (Back) [8]**

ANSWER

Algorithm:

1. Load n → reg_n
2. sum ← 0, i ← 0
3. While i < n: load next input x; sum ← sum + x; i ← i + 1
4. mean ← sum / n
5. Output mean, assert done

Mean of N Numbers Processor — FSMD State Table		
State	**Condition**	**Operation**
S0	start=1	reg_n←n, sum←0, i←0 → go to S1
S1	i = reg_n	mean←sum/reg_n → go to S2
S1	i < reg_n	sum←sum+x_in, i←i+1 → stay S1 (load next x on each cycle)
S2	—	output←mean, done←1 → go to S0

Datapath: Registers: reg_n, sum, i, mean. ALU: adder (sum+x), counter (i+1), comparator (i=n), divider (sum/n). Controller: ld_n, ld_sum, ld_i, ld_mean, done.

Q7. Develop algorithm; draw the state diagram and design FSMD for finding the largest of 4 integers.	**2073 Bhadra (Regular) | 2071 Ashwin (Back) [8]**

ANSWER

Algorithm:

1. Load a, b, c, d
2. max ← a
3. If b > max: max ← b
4. If c > max: max ← c
5. If d > max: max ← d
6. Output max, done

Largest of 4 Integers — FSMD State Table		
State	**Condition**	**Operation**
S0	start=1	a←in_a, b←in_b, c←in_c, d←in_d, max←in_a → go to S1
S1	b > max	max←b → go to S2
S1	b ≤ max	→ go to S2
S2	c > max	max←c → go to S3
S2	c ≤ max	→ go to S3
S3	d > max	max←d → go to S4
S3	d ≤ max	→ go to S4
S4	—	output←max, done←1 → go to S0

Optimization: States S1–S3 can be merged using a single comparator with MUX'd inputs to save hardware.

Datapath: Registers: a, b, c, d, max. ALU: comparator (> operator). MUX to select current comparison input.

Q8. Design a dual-purpose processor that takes n numbers as input and outputs both their average and maximum value.	**2079 Chaitra (Regular) [8]**

ANSWER

Algorithm:

1. Load n → reg_n
2. sum ← 0, i ← 0, max ← −∞ (or first input)
3. While i < n: load x; sum ← sum + x; if x > max then max ← x; i ← i + 1
4. avg ← sum / n
5. Output avg and max, done

Average + Maximum Dual Processor — FSMD State Table		
State	**Condition**	**Operation**
S0	start=1	reg_n←n, sum←0, i←0 → load x[0], max←x[0] → go to S1
S1	i = reg_n	avg←sum/reg_n → go to S2
S1	i < reg_n	sum←sum+x_in; if x_in>max then max←x_in; i←i+1 → stay S1
S2	—	out_avg←avg, out_max←max, done←1 → go to S0

Datapath: Shares one adder for both accumulation (sum) and index counting. A dedicated comparator updates max in parallel with adder — uses concurrency to compute sum and max update simultaneously.

Q9. Design a single purpose processor to compute sum of digits of a given number.	**2076 Baisakh (Back) [8]**

ANSWER

Algorithm:

1. Load N → reg_n
2. sum ← 0
3. While reg_n > 0: digit ← reg_n mod 10; sum ← sum + digit; reg_n ← reg_n / 10
4. Output sum, done

Sum of Digits Processor — FSMD State Table		
State	**Condition**	**Operation**
S0	start=1	reg_n←N, sum←0 → go to S1
S1	reg_n = 0	output←sum, done←1 → go to S0
S1	reg_n > 0	digit←reg_n mod 10, sum←sum+digit, reg_n←reg_n/10 → stay S1

Datapath: Registers: reg_n, sum, digit. ALU: modulo-10 unit (reg_n mod 10), adder (sum+digit), divide-by-10 unit (reg_n/10), comparator (reg_n=0).

Q10. Design a custom single-purpose processor to compute factorial of a number n (n!).	**Model Question | 2075 Chaitra (Back) [8]**

ANSWER

Algorithm:

1. Load n → reg_n
2. result ← 1
3. While reg_n > 1: result ← result × reg_n; reg_n ← reg_n − 1
4. Output result, done

Factorial Processor — FSMD State Table		
State	**Condition**	**Operation**
S0	start=1	reg_n←n, result←1 → go to S1
S1	reg_n ≤ 1	output←result, done←1 → go to S0
S1	reg_n > 1	result←result×reg_n, reg_n←reg_n−1 → stay S1

Datapath:

- Registers: reg_n (counter, initialized to n), result (accumulator, init to 1)

- ALU: multiplier (result × reg_n), decrementer (reg_n − 1), comparator (reg_n ≤ 1)
- Controller signals: ld_n, ld_result, dec_n (decrement enable), mul_en, done

Optimization — Concurrency: The decrement (reg_n ← reg_n − 1) and the multiplication (result ← result × reg_n) are independent and can be initiated in the same state, reducing latency if pipelined.

Q11. Design a custom processor to compute sum of squares of n numbers: S = $1^2 + 2^2 + ... + n^2$	**Model Question | 2074 Ashwin (Back) [8]**

ANSWER

Algorithm:

1. Load n → reg_n
2. sum ← 0, i ← 1
3. While i ≤ reg_n: sum ← sum + (i × i); i ← i + 1
4. Output sum, done

Sum of Squares Processor — FSMD State Table

State	Condition	Operation
S0	start=1	reg_n←n, sum←0, i←1 → go to S1
S1	i > reg_n	output←sum, done←1 → go to S0
S1	i ≤ reg_n	sq←i×i, sum←sum+sq, i←i+1 → stay S1

Datapath:

- Registers: reg_n (input), i (loop counter, init 1), sq (intermediate i^2), sum (accumulator)
- ALU: squarer (i×i), adder (sum+sq), incrementer (i+1), comparator (i > reg_n)
- Optimization: sq←i×i and incrementer i←i+1 can operate concurrently (different hardware units).

Q12. What is the difference between a custom single-purpose processor and a general-purpose processor? When would you choose each?	**Model Question | 2076 Chaitra (Back) [4+4]**

ANSWER

Property	Custom Single-Purpose Processor	General-Purpose Processor
Function	Fixed — executes one specific algorithm	Programmable — executes any program
Hardware	Fully custom datapath + controller	Standard CPU architecture
Performance	Optimal — hardware perfectly matched to task	Lower (overhead from ISA generality)

Power	Lowest (no wasted logic)	Higher (pipeline, caches, decoders)
Cost	High NRE, low unit cost (high volume)	Low NRE, moderate unit cost
Flexibility	None — algorithm fixed in hardware	Complete — change program anytime
Time-to-Market	Long (custom design needed)	Short (off-the-shelf chip)
Examples	AES crypto engine, JPEG encoder	ARM Cortex-M, x86

When to Choose:

Custom Processor: When performance or power is critical and the algorithm is fixed. High-volume products where unit cost savings justify NRE. Examples: video codec ASICs, network packet processors, image sensor ISPs.

GPP (Microcontroller): When flexibility, fast development, and low NRE are important. Algorithm may change. Low to medium volume products. Examples: industrial controllers, smart home devices, consumer electronics.

Summary: Topics Covered in Chapter 2

S.N.	Topic/Processor Design	Exam(s) Asked	Marks
1	GCD Processor (subtraction method)	2081 Chaitra, 2079 Chaitra, 2078 Chaitra, 2077 Baisakh, 2072 Magh	8
2	LCM Processor	2082 Kartik, 2075 Baisakh	8
3	x^n (Power) Processor	2080 Chaitra, 2075 Baisakh	8
4	Fibonacci Processor + Optimization	2080 Ashwin, 2074 Bhadra	5+3
5	Mean of N Numbers	2081 Ashwin, 2076 Jestha	8
6	Largest of 4 Integers	2073 Bhadra, 2071 Ashwin	8
7	Average + Maximum (Dual-Purpose)	2079 Chaitra	8
8	Sum of Digits	2076 Baisakh	8
9	Optimization Techniques (4 methods)	2070 Bhadra, 2077 Ashwin	8
10	Factorial Processor	2075 Chaitra, Model Q	8
11	Sum of Squares Processor	2074 Ashwin, Model Q	8
12	Custom vs GPP — Comparison & Selection	2076 Chaitra, Model Q	4+4